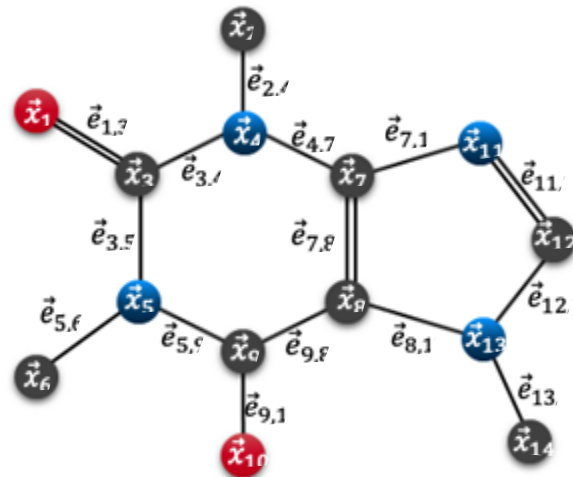


Graph neural Networks for small molecules

Representation of small molecules as graphs

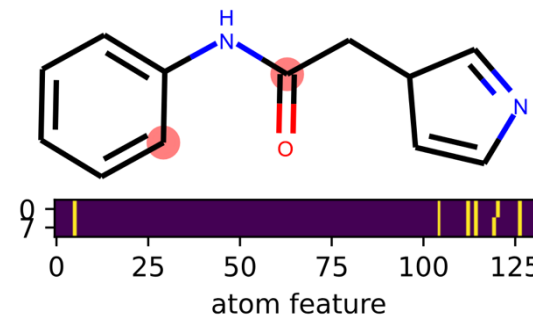
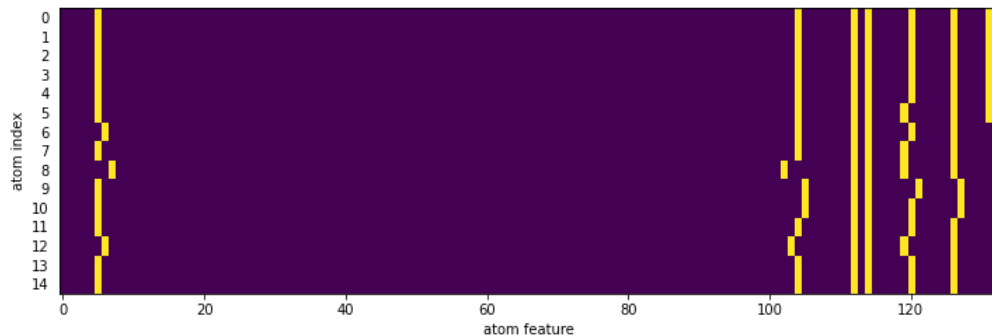
- Atoms can be interpreted as the nodes of a graph and bonds as the edges of a graph
- All N atoms have to be numbered arbitrarily
- Node feature vectors $\vec{x}_i \in \mathbb{R}^{D_n}$ for node i are calculated and saved in a matrix $X \in \mathbb{R}^{N \times D_n}$
- Bond feature vectors $\vec{e}_{i,j} \in \mathbb{R}^{D_e}$ (bond between atom i and j) are calculated and saved in a tensor $E \in \mathbb{R}^{N \times N \times D_e}$
- Adjacency matrix $A \in \mathbb{R}^{N \times N}$ stores between which atoms a bond exists:
 - $A_{i,j} = 1 \rightarrow$ a bond exists between atom i and j
 - $A_{i,j} = 0 \rightarrow$ no bond exists between atom i and j
- X , E , and A are used as input for a graph neural network (GNN)



$$\vec{x}_i = \begin{pmatrix} \text{atom type} \\ \text{mass} \\ \text{valence} \\ \text{charge} \\ \dots \end{pmatrix} \quad \vec{e}_{i,j} = \begin{pmatrix} \text{bond type} \\ \text{part of ring} \\ \dots \\ \dots \\ \dots \end{pmatrix}$$

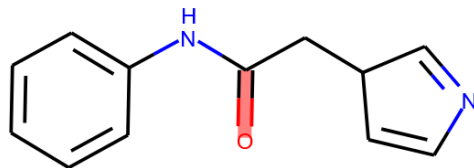
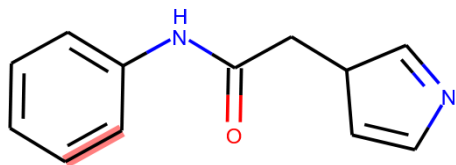
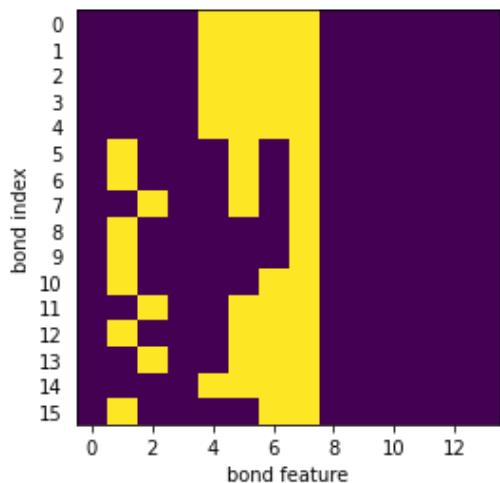
Atom features (example)

- One-hot encoding of atomic number, degree, formal charge, chirality, number of attached hydrogens, hybridization.
- In this example, atom index 0 and 7 are very similar but differ in terms of aromaticity and the number of hydrogens attached.

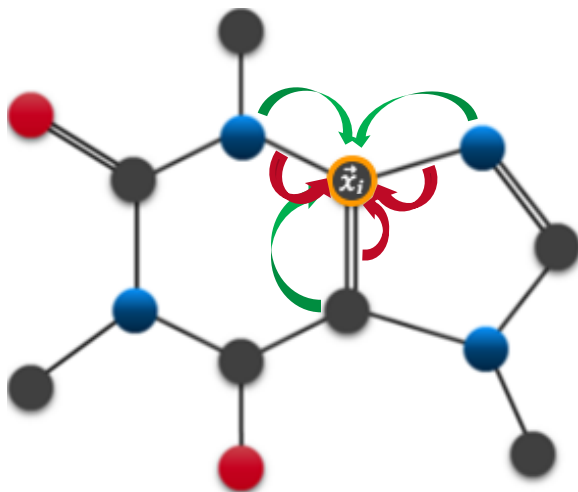


Bond features (example)

- One-hot encoding of bond type (single, double, triple, aromatic), whether the bond is conjugated or not, whether it is part of a ring or not and stereochemistry (none, unspecified, cis, trans).



Typical forward pipeline of a GNN



- First, the node feature vectors are iteratively updated T times by using information about neighboring atoms and edges:

$$\vec{x}_i = \vec{x}_i^{(0)} \rightarrow \vec{x}_i^{(1)} \rightarrow \dots \rightarrow \vec{x}_i^{(T)} \in \mathbb{R}^{D_n}$$

- Update for a single atom feature vector ($N(i)$ is the set of neighboring atoms):

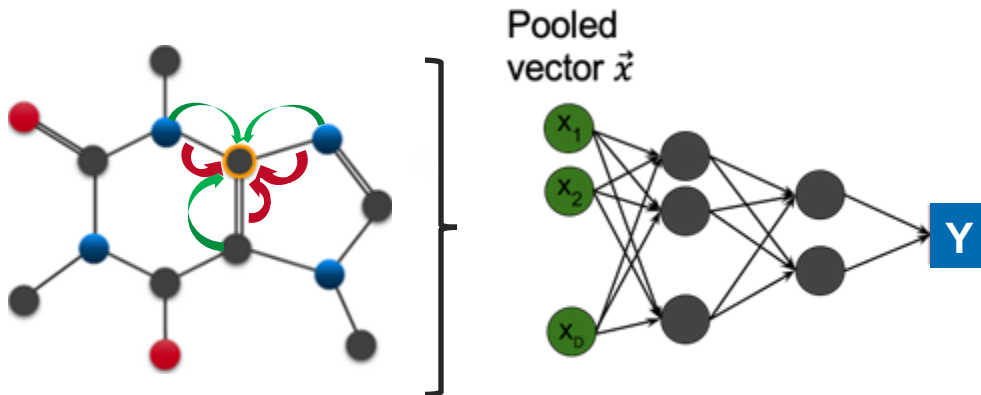
$$\vec{x}_i^{(t+1)} = \sum_{j \in N(i)} U_t(\vec{x}_i^{(t)}, \vec{x}_j^{(t)}, \vec{e}_{i,j})$$

- After T update steps, we pool all feature vectors $\vec{x}_i$:

$$\text{Pooling}(\vec{x}_1^{(T)}, \vec{x}_2^{(T)}, \dots, \vec{x}_N^{(T)}) = \vec{x} \in \mathbb{R}^{D_n}$$

- Predicting a molecule/graph property:

$$\text{FFN}(\vec{x}) = y \in \mathbb{R}$$

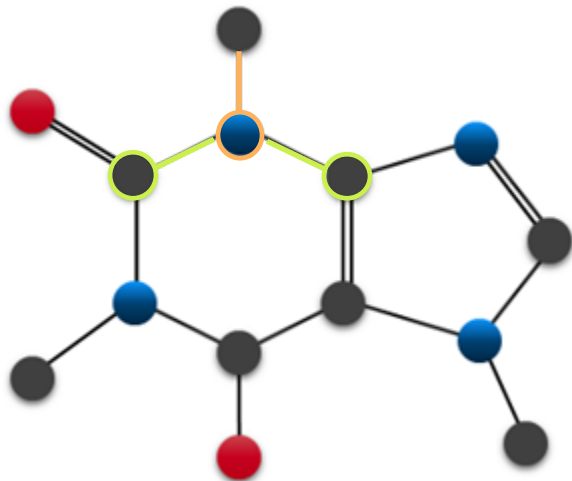


- Whole network is trained end-to-end
 - Parameters of the update functions and the FFN are adjusted simultaneously
 - Training is the same as for standard deep neural networks

- GNNs can also be pre-trained in a self-supervised way
 - For example: Masking nodes and prediction the type of node from the neighboring bond and node feature vectors

- **Vanilla GNN** face some problems when modelling molecules:
 - **Tottering**: depending on the graph structure, messages can go back and forth introducing noise and wasting computations.
 - **Node (atom) centric representation**: edge information is used only during message passing phase.
 - **Oversmoothing**: too many message-passing steps can make atom embeddings all too similar.
- Directed Message Passing Neural Networks incorporate domain specific tweaks to mitigate this problems:
 - Use **directed graphs** to pass messages, avoiding tottering and mitigating oversmoothing.
 - Use an **edge centric representation** (one hidden state vector per directed edge).
 - **Keep edge (bond) information** more explicitly.

Directed Message Passing Neural Network (D-MPNN)



- Calculating atom feature vectors $\vec{x}_i \in \mathbb{R}^{D_n}$ and bond feature vectors $\vec{e}_{i,j} \in \mathbb{R}^{D_e}$

- Initial hidden representations of atom-bond-pairs:

$$\vec{h}_{i,j}^{(0)} = \text{ReLU}(W_h \cdot \text{concat}(\vec{x}_i, \vec{e}_{i,j}))$$

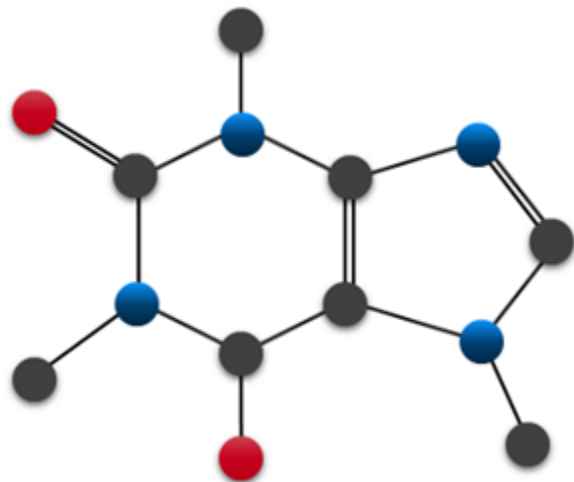
- Update hidden vectors T times

$$\vec{m}_{i,j}^{(1)} = \sum_{k \in N(i) \setminus j} \vec{h}_{k,i}^{(0)} \quad \vec{h}_{i,j}^{(1)} = \text{ReLU}(\vec{h}_{i,j}^{(0)} + W_m^1 \cdot \vec{m}_{i,j}^{(1)})$$

$$t = 2: \quad \dots$$

$$t = T: \quad \vec{m}_{i,j}^{(T)} = \sum_{k \in N(i) \setminus j} \vec{h}_{i,k}^{(T-1)} \quad \vec{h}_{i,j}^{(T)} = \text{ReLU}(\vec{h}_{i,j}^{(0)} + W_m^T \cdot \vec{m}_{i,j}^{(T)})$$

Directed Message Passing Neural Network (D-MPNN)



- Initial hidden representations of node-bond-pairs:

$$\vec{h}_{i,j}^{(0)} = \text{ReLU}(W_h \cdot \text{concat}(\vec{x}_i, \vec{e}_{i,j}))$$

- Update hidden vectors T times

For $t = 1 \dots T$:

$$\vec{m}_{i,j}^{(t)} = \sum_{k \in N(i) \setminus j} \vec{h}_{k,i}^{(t-1)} \quad \vec{h}_{i,j}^{(t)} = \text{ReLU}(\vec{h}_{i,j}^{(0)} + W_m^t \cdot \vec{m}_{i,j}^{(t)})$$

- Calculating atom representations for all atoms i :

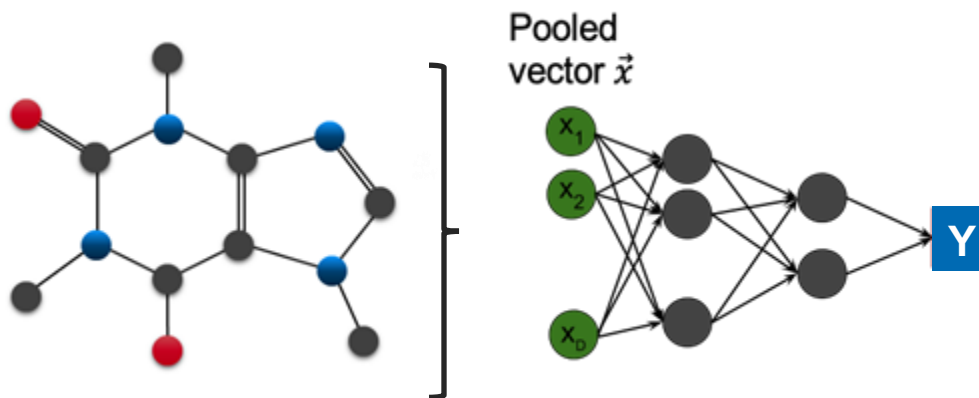
$$\vec{m}_i = \sum_{k \in N(i)} \vec{h}_{i,k}^{(T)} \quad \vec{h}_i = \text{ReLU}(W_a \cdot \text{concat}(\vec{x}_i, \vec{m}_i))$$

- Pooling all atom feature vectors $\vec{x} = \sum_{i=1, \dots, N} \vec{h}_i$

- Predicting a molecule/graph property:

$$\text{FFN}(\vec{x}) = y \in \mathbb{R}$$

After model training



- We can use this model as our final prediction model
- Alternatively, we can extract the vectors $\vec{x}$ for all molecules and train another ML model for the prediction task with these vectors as model input

Comparison to Transformer Networks

	BACE <i>RMSE</i>	Clearance <i>RMSE</i>	Delaney <i>RMSE</i>	Lipo <i>RMSE</i>	BACE <i>ROC</i>	BBBP <i>ROC</i>	ClinTox <i>ROC</i>	SR-p53 <i>ROC</i>
D-MPNN	2.253	49.754	1.105	1.212	0.812	0.697	0.906	0.719
ChemBERTa-2								
MTR-10M	1.417	48.934	0.858	0.744	0.783	0.733	0.601	0.827
MTR-77M	1.363	48.515	0.889	0.798	0.799	0.728	0.563	0.817

Dataset	BBBP	Tox21	ClinTox	HIV	BACE	SIDER
Tasks	1	12	2	1	1	27
D-MPNN ⁵⁴	71.2	68.9	90.5	75.0	85.3	63.2
MoLFORMER-XL	93.7	84.7	94.8	82.2	88.21	69.0

Comparison to Transformer Networks

Measure	Graph-Based			Geometry-Based			SMILES-Based	
	A-FP	123-gnn	GC	CM	DTNN	MPNN	MOLFORMER-XL	ChemBERTa
α	0.492	0.27	1.37	0.85	0.95	0.89	0.3327	0.8510
C_v	0.252	0.0944	0.65	0.39	0.27	0.42	0.1447	0.4234
G	0.893	0.0469	3.41	2.27	2.43	2.02	0.3362	4.1295
gap	0.00528	0.0048	0.01126	0.0086	0.0112	0.0066	0.0038	0.0052
H	0.893	0.0419	3.41	2.27	2.43	2.02	0.2522	4.0853
ϵ_{homo}	0.00358	0.00337	0.00716	0.00506	0.0038	0.00541	0.0029	0.0044
ϵ_{lumo}	0.00415	0.00351	0.00921	0.00645	0.0051	0.00623	0.0027	0.0041
μ	0.451	0.476	0.583	0.519	0.244	0.358	0.3616	0.4659
$\langle R^2 \rangle$	26.839	22.90	35.97	46.00	17.00	28.5	17.0620	86.150
U_0	0.898	0.0427	3.41	2.27	2.43	2.05	0.3211	3.9811
U	0.893	0.111	3.41	2.27	2.43	2.00	0.2522	4.3768
ZPVE	0.00207	0.00019	0.00299	0.00207	0.0017	0.00216	0.0003	0.0023
Avg MAE	2.6355	1.9995	4.3536	4.7384	2.3504	3.1898	1.5894	8.7067
Avg std MAE	0.0854	0.0658	0.1683	0.1281	0.1008	0.1108	0.0567	0.1413

Table 8. MOLFORMER performance on QM9 test set. Our best MOLFORMER variant is pre-trained on PubChem+ZINC dataset and fine-tuned for each measure. Baseline performance values are taken from^{28,40,62}. **Blue** and **Orange** indicates best and second-best performing model, respectively. MOLFORMER trained with rotary embeddings on PubChem+ZINC achieves the best Avg MAE and Avg. std. MAE across all tasks.

Comparison to Transformer Networks

Transformer encoder	D-MPNN
Ingests raw SMILES .	Requires feature engineering .
Weak chemical prior .	Strong chemical prior .
Needs to learn molecular grammar (rings, branches, etc.).	Knows atoms and bonds by construction.
Must infer connectivity and structure , could recover 3D information.	Explicit 2D structure .
Needs big datasets/augmentation to shine.	Good performance with small/medium datasets .
$O(\text{sequence_length}^2)$	$O(V + E)$
All-to-all attention from layer 1 .	Struggles with long-range dependencies .
Single architecture can process many inputs (proteins and reactions).	Not possible / requires adaptations like difference encoding or graph merging.